🧭 중간 점검 — 지금까지(Day 1~5) 복습 + 앞으로(Day 6~14) 미리보기

> 자신 없을 때 펴보는 "지도" 문서. 외우는 게 목적이 아니라, **"내가 지금 전체 중 어디에 있고, 이게 왜 필요한지"**를 잡는 게 목적. 읽다가 막히면 그 부분만 Claude한테 "이 개념 다시 설명" 하면 돼.

PART A. 지금까지 뭘 만들었나 (Day 1~5 복습)

A-0. 한 문장으로

"강사가 아이 정보·수업기록을 서버에 넣고(POST) → 필요할 때 꺼내보고(GET) → AI가 그걸 읽고 피드백을 써주는 웹 서버."

지금까지 만든 건 딱 이거야. 화면(웹페이지)은 아직 없고, **Swagger**(자동 생성되는 API 테스트 화면)로 눌러보는 단계까지 왔어.

A-1. 제일 중요한 것 — "한 번의 요청이 흐르는 길"

이거 하나만 머리에 있으면 코드가 안 무서워져. 나머지는 전부 이 길 위의 부품이야.

```
손님 요청
   │
   ▼
① main.py        → "이 주소는 어느 라우터로 보낼까?" (교통정리)
   ▼
② router 파일     → 실제 처리 (넣기/꺼내기 로직)
   ▼
③ database.py    → DB 문 열기 (get_db)
   ▼
④ models.py      → DB에서 조회/저장 (테이블 설계도)
   ▼
⑤ schemas.py     → 주고받는 데이터 모양 검사 + 정리
   ▼
   JSON 응답
```

말로 외우는 버전:

> main(어디로) → router(처리) → database(연결) → models(DB) → schemas(모양) → 응답

A-2. 파일이 왜 5개나 있나 (역할 = 식당 직원)

파일	식당 비유	하는 일	왜 따로 있나
`main.py`	안내데스크	앱 켜기 + 요청을 라우터로 연결	입구는 하나여야 하니까
`routers/*.py`	주방 코너	요청 실제 처리	메뉴(기능)별로 나눠야 안 엉킴
`models.py`	창고 선반 설계도	DB 테이블 모양	DB에 뭐가 저장되는지의 "진짜 모양"
`schemas.py`	주문서/영수증 양식	주고받는 데이터 검사	이상한 데이터 걸러냄
`database.py`	창고 문지기	DB 연결 열고 닫기	연결 관리는 한 곳에서

핵심: 파일이 많은 건 어려워서가 아니라 **"역할별로 쪼개야 관리가 쉬워서"**야. 한 파일에 다 넣으면 수백 줄짜리 괴물이 됨. → 이게 네가 day2, day3 문서에 이미 쓴 내용이야. 잘 잡았어.

A-3. ⭐ 진짜 핵심 — "4단계 반복 패턴"

Day 2, 3, 5는 사실 **똑같은 일을 대상만 바꿔서 반복**한 거야. 새 기능이 생길 때마다 이 4단계가 똑같이 돌아가.

```
① models.py   → 테이블 만들기      (이 데이터를 어디에 저장할지)
② schemas.py  → 데이터 양식 만들기  (뭘 받고 뭘 돌려줄지)
③ router 파일  → API 만들기         (POST 넣기 / GET 꺼내기)
④ main.py     → 라우터 연결         (import + include_router 2줄)
```

대상만 바뀜:

- Day 2 = 대상이 **아이(Child)**
- Day 3 = 대상이 **수업기록(LessonRecord)**
- Day 5 = 대상이 **피드백(Feedback)** + AI 호출이 끼어듦

> 💡 자신감 포인트: 너는 이 패턴을 이미 **세 번** 돌렸어. Day 6~14에도 이 패턴이 계속 나와. 새로운 게 나와도 "아 이거 그 4단계구나" 하고 보면 돼.

A-4. Day별로 뭘 했는지 (한 줄씩)

Day	대상	핵심 한 일	새로 배운 개념
Day 1	환경	서버 켜기 + /health	FastAPI 켜는 법
Day 2	아이	POST 등록 / PATCH 수정	POST vs PATCH, `model_dump`, `setattr`
Day 3	수업기록	POST 저장 / GET 조회(최근 5개)	`relationship`, `filter·order_by·limit`, 기본키/외래키
Day 4	AI 연결	프롬프트 조립 + Gemini 호출	프롬프트, f-string, API/API키, 매개변수

Day	대상	핵심 한 일	새로 배운 개념
Day 5	피드백	AI 결과를 DB 저장 + GET 조회 + 404	`json.loads`, `List[...]`, `HTTPException`

A-5. 🧠 자가 진단 — "머리에 들어있나" 체크 (안 보고 대답)

아래 질문에 **파일 안 보고 30초씩 말로** 대답해봐. 막히는 게 곧 다음 복습 대상이야. (3개 이상 막히면 그 Day 문서를 다시 보면 됨. 못 막혀도 정상이야.)

기초 흐름

1. 손님이 `GET /children/3` 요청하면 파일들을 어떤 순서로 지나가?
2. `main.py`에 라우터를 "만들기만" 하면 왜 작동 안 해? 뭘 2줄 더 써야 해?

models vs schemas (제일 헷갈리는 것) 3. `models.py`랑 `schemas.py`는 뭐가 달라? (힌트: 창고 선반 vs 주문서) 4. `Create` 스키마랑 `Response` 스키마는 뭐가 달라? Response엔 보통 뭐가 더 붙어?

DB 다루기 5. POST 저장 3종 세트는? (`db.___()` → `db.___()` → `db.___()`) 6. `.filter(LessonRecord.child_id == child_id)`에서 왼쪽과 오른쪽은 각각 뭐야? 7. `.all()`이랑 `.first()`는 언제 각각 써? 8. `id`(기본키)랑 `child_id`(외래키)의 차이는?

AI 부분 (Day 4~5) 9. `prompt_service.py`랑 `llm_service.py`는 역할이 어떻게 달라? (힌트: 주문서 작성 vs 주방에 주문 넣기) 10. `load_dotenv`랑 `client`는 왜 함수 **밖**에 뒀어? 11. `json.loads(result)`는 뭘 뭐로 바꿔? 왜 그게 필요해? 12. `HTTPException(status_code=404)`는 언제, 왜 일부러 발생시켜?

> ✅ 완료 기준: 노트 양이 아니라 **"내 말로 30초 설명 가능한지"**. 3, 5, 6, 9번은 이 프로젝트의 뼈대라 꼭 말할 수 있어야 해.

PART B. 앞으로 뭘 할까 (Day 6~14 미리보기)

전체를 3덩어리로 보면 돼:

```
[가벼움]  Day 6~7   1주차 마무리 (테스트 + 문서)      ← 지금까지 한 걸 정리
[고비]    Day 8~11  RAG (AI를 똑똑하게)              ← 새 개념, 여기가 핵심 고비
[마감]    Day 12~14 Streamlit (화면 만들기)          ← 눈에 보이는 결과물
```

B-1. Day 6~7 — 1주차 마무리 (가벼움)

Day 6: 통합 테스트 — "왜 필요한가"

지금까지 기능을 하나씩 따로 만들었지? **한 번도 처음부터 끝까지 쭉 돌려본 적이 없어.**

- 아이 3명(위축형/승부욕형/평범형) 등록 → 각자 수업기록 몇 개 → 피드백 생성
- 목적: **"성향이 다르면 피드백 톤도 다르게 나오나?"** 실제로 눈으로 확인.
- 코드 새로 안 짜. **이미 만든 API를 순서대로 눌러보는 것.** (Swagger에서 클릭)

> 왜 중요하냐면: 부품이 다 있어도 **조립했을 때 진짜 굴러가는지는 별개**야. 여기서 숨은 버그가 튀어나옴.

Day 7: README — "왜 필요한가"

`README.md` = 이 프로젝트를 처음 보는 사람(그리고 3주 뒤의 너)을 위한 설명서.

- **API 명세**: 어떤 주소로 뭘 보내면 뭐가 나오는지 목록
- **ERD**: 테이블 3개(children ─ lesson_records ─ feedbacks)가 어떻게 연결됐는지 그림
- **회고**: 배운 것 / 어려웠던 것

> 왜 중요하냐면: 이거 쓰다 보면 **"내가 뭘 만들었는지"가 머릿속에서 정리돼.** 남 보여주기 전에 나를 위한 정리야.

B-2. Day 8~11 — RAG (여기가 진짜 고비 — 근데 겁먹을 필요 없음)

> 이 섹션은 "읽고 나면 자신 있게 시작할 수 있게" 최대한 자세히 풀었어. 한 번에 다 외우지 마. **흐름 그림 하나 + 지도 비유 하나**만 잡으면 나머진 따라와.

🎯 B-2-0. 왜 이걸 하는가 — 실제 피드백으로 비교 (동기부터)

지금 우리 AI는 **아이 정보 + 수업기록**만 보고 답을 써. 그래서 이런 느낌이야:

[RAG 없는 지금]

> "유리가 오늘 줄넘기 20개를 했어요. 잘했어요! 다음엔 더 열심히 해봐요."

→ 틀린 말은 아닌데 **뻔하고 근거가 없어.** 강사가 "이걸 학부모한테 그대로 보내긴 좀..." 싶은 수준.

[RAG 붙인 후] — AI가 "아동 칭찬 심리" 참고자료를 같이 읽고 씀:

> "유리가 20개를 성공한 것보다 **'중간에 걸려도 다시 시도한 태도'를 구체적으로 칭찬**해주세요. 위축형 아이는 결과보다 과정 칭찬이 자신감에 효과적입니다(참고자료 기반)."

→ **전문성이 생김.** 이 차이를 만드는 게 RAG야. "AI를 더 똑똑하게" = 이 프로젝트의 하이라이트.

🧠 B-2-1. RAG 한 문장 + 오픈북 비유

RAG = "AI한테 답 시키기 전에, 관련 참고자료를 먼저 찾아서 프롬프트에 끼워주는 것."

- 지금 AI = **아무것도 안 보고 시험 보는 학생** → 아는 것만, 뻔하게 답함
- RAG = **오픈북 시험 학생** → 관련 페이지를 펴서 보고 답함 → 정확하고 근거 있음

우리가 하는 일은 딱 하나: **"AI가 시험 볼 때 펴볼 책을 준비하고, 문제에 맞는 페이지를 대신 찾아서 책상에 올려주는 것."**

🗺️ B-2-2. 제일 헷갈리는 "임베딩·벡터" — 지도 비유로 확실하게

여기서 대부분 겁먹는데, 비유 하나면 끝나.

핵심 질문: 컴퓨터는 "칭찬"이랑 "격려"가 비슷한 말인 걸 어떻게 알까? → 컴퓨터는 글 뜻을 몰라. **그래서 모든 문장을 "지도 위의 점"으로 바꿔놔.**

```
📍 "결과보다 과정을 칭찬하라"
📍 "아이를 격려하는 법"          ← 뜻이 비슷하면 서로 가까이 찍힘

                    📍 "줄넘기 줄 길이 조절법"   ← 뜻이 다르면 멀리 찍힘
```

- **임베딩(Embedding)** = "문장 → 지도 위 좌표(점)로 바꾸는 것"
- **벡터(Vector)** = 그 좌표값 (실제론 숫자 여러 개의 목록. 예: `[0.12, -0.83, 0.44, ...]`)
- **비슷함 찾기** = 지도에서 **점끼리 거리가 가까운지** 재는 것. 가까우면 = 뜻이 비슷함.

> 💡 핵심 안심: **점으로 바꾸는 계산(임베딩)은 AI 라이브러리가 다 해줘.** 너는 숫자를 손으로 만들 일 없어. 너는 그냥 `embed(문장)` 같은 함수를 부르면 좌표가 툭 나와. **원리만 "지도에 점 찍고 가까운 거 찾기"로 이해하면 끝.**

검색이 되는 원리 (한 문장):

> 질문("유리 피드백 써줘")도 똑같이 점으로 찍고 → 그 점에서 **제일 가까운 자료 점 3개**를 집어온다.

🔄 B-2-3. 우리 프로젝트 실제 데이터로 흐름 따라가기

참고자료 파일에 이런 문장들이 있다고 하자 (`아동칭찬가이드.txt`):

```
- 위축형 아이는 결과보다 과정을 칭찬해야 자신감이 붙는다.
- 승부욕이 강한 아이는 도전적 목표를 주면 동기가 올라간다.
- 줄넘기는 손목 스냅으로 돌려야 팔이 덜 아프다.
```

[준비 단계 — 프로그램 켤 때 딱 1번]

```
① 이 파일을 읽는다
② 문장 단위로 쪼갠다(Chunking)  → 조각 3개
③ 각 조각을 점으로 바꾼다(Embedding) → 좌표 3개
④ 벡터 DB에 저장 → 이제 "검색 가능한 참고자료 지도" 완성
```

[실행 단계 — 유리 피드백 만들 때]

```
⑤ "위축형 유리 피드백" 이라는 질문도 점으로 바꾼다
⑥ 지도에서 가까운 조각 찾기 → "위축형은 과정을 칭찬" 조각이 딱 걸림
     (줄넘기 손목 스냅 조각은 멀어서 안 걸림 → 지금 질문과 상관없으니까 정확히 걸러짐!)
⑦ 걸린 조각을 프롬프트의 rag_context 자리에 끼운다   ← ⭐
⑧ AI가 그 조각까지 읽고 피드백 작성 → B-2-0의 "RAG 붙인 후" 결과가 나옴
```

> 이게 RAG 전부야. **"쪼개고 → 점 찍어 저장 → 질문과 가까운 점 찾아 → 프롬프트에 끼움."**

🧩 B-2-4. 왜 "그냥 자료 전부를 프롬프트에 넣으면" 안 돼?

- 자료가 수십 페이지면 **프롬프트에 다 못 넣어** (AI가 한 번에 읽는 양 한계 + 돈·속도 낭비)
- 다 넣으면 상관없는 내용(줄넘기 손목 스냅)까지 섞여서 **답이 산만해짐**
- 그래서 **"이 질문에 필요한 조각만 골라 넣기"** = RAG가 존재하는 이유

🗓️ B-2-5. Day별로 정확히 뭘 하나 + 네가 칠 것 vs Claude가 깔 것

CLAUDE.md 규칙: **어려운 뼈대·보일러플레이트는 Claude가 깔고, 핵심 개념 줄은 네가 손으로 친다.**

Day	하는 일	🖐️ 네가 직접 칠 핵심	🤖 Claude가 깔아줄 것
Day 8	참고자료 준비 + 쪼개기	자료 내용 고르기 / chunk 나누는 기준 정하기	파일 읽는 코드, 라이브러리 설치
Day 9	임베딩 + 벡터 저장 + 검색	`embed()` 부르고 `search()` 불러 결과 꺼내는 줄	벡터 DB 세팅, 연결 보일러플레이트
Day 10	피드백에 RAG 연결	`build_prompt`의 `rag_context` 자리에 검색 결과 끼우기	함수들 이어주는 뼈대
Day 11	다듬기 + 확인	"이 조각이 잘 걸렸나" 눈으로 검증	테스트 헬퍼

> 즉 **네 몫은 "함수 부르고 결과를 프롬프트에 끼우는 줄"** 정도야. 넌 이미 Day 4~5에서 `build_prompt()`, `generate_feedback()` 함수 부르고 결과 넘기는 걸 **해봤어.** 같은 근육이야.

😌 B-2-6. 시작 전 자신감 체크 (안심 포인트 4개)

1. **새 프로젝트 아님.** Day 4 `prompt_service.py`에 네가 남긴 `rag_context` 빈칸 — **그걸 채우는 거야.** (네가 day4 문서에 "`rag_context`는 Day 8 거"라고 직접 적어놨어. 미래의 너한테 예약해둔 자리.)
2. **패턴 재활용.** A-3의 4단계, Day 4~5의 "함수 부르고 결과 넘기기"가 그대로 반복돼.
3. **숫자 계산은 안 함.** 임베딩·거리계산은 라이브러리가 처리. 너는 흐름과 함수 호출만.
4. **개념부터 시작함.** 백지에서 "해봐" 절대 안 해. Day 8 아침에 이 문서 다시 읽고, 개념 확인부터 감.

RAG 시작 전 이거 3개만 말로 설명되면 준비 완료:

- (a) RAG를 오픈북 비유로 30초
- (b) 임베딩 = 뭘 뭐로 바꾸는 것? (힌트: 문장 → 지도 위 점)
- (c) `rag_context` 자리에 뭐가 들어가고, 그게 왜 필요해?

📖 B-2-7. 미리 알아두면 든든한 용어 사전 (한 줄씩)

용어	한 줄 뜻	우리 프로젝트에선
RAG	참고자료 찾아서 프롬프트에 끼워 답 시키기	피드백을 전문적으로
Chunk(청크)	긴 문서를 자른 작은 조각	"위축형은 과정 칭찬" 한 문장
Chunking	문서를 조각으로 쪼개는 작업	자료 → 문장 단위로 자름
Embedding(임베딩)	문장을 지도 위 점(숫자)으로 바꿈	`embed("...")` → 좌표

용어	한 줄 뜻	우리 프로젝트에선
Vector(벡터)	그 점의 좌표값(숫자 목록)	`[0.1, -0.8, ...]`
Vector DB(벡터DB)	점들을 저장하고 "가까운 거" 찾아주는 창고	참고자료 지도 보관소
Retrieval(검색)	질문과 가까운 조각 찾아오기	유리 질문 → 관련 조각 3개
rag_context	찾아온 조각을 담아 프롬프트에 끼우는 자리	Day 4에 만들어둔 빈칸

B-3. Day 12~14 — Streamlit (눈에 보이는 결과물, 마감)

Streamlit이 뭐냐 & 왜 필요한가

지금은 **Swagger**(개발자용 테스트 화면)로만 눌러봤지? 남한테 보여주긴 못생겼어. **Streamlit = 파이썬 코드 몇 줄로 웹 화면(버튼·입력창·표)을 만드는 도구.**

- HTML/CSS 몰라도 됨. `st.button("생성")` 같은 파이썬 함수로 화면이 생김.
- 목적: 강사가 **브라우저에서 아이 고르고 → 버튼 누르면 → 피드백이 화면에 뜨는** 진짜 앱 모양.

흐름 (지금까지 만든 걸 화면에 연결)

```
[Streamlit 화면]  강사가 아이 선택 + 버튼 클릭
      │
      ▼
   우리가 만든 API 호출 (POST /feedbacks/generate)   ← Day 5에서 만든 그거!
      │
      ▼
   돌아온 피드백을 화면에 예쁘게 표시 (st.write 등)
```

> 안심 포인트: Streamlit은 **새 로직이 아니라 "이미 만든 API에 껍데기(화면)를 씌우는 것"**이야. 뒤(백엔드)는 다 됐고, 앞(화면)만 붙이는 마감 작업.

- Day 12: Streamlit 기본 (화면 요소 몇 개)
- Day 13: 화면 ↔ 우리 API 연결
- Day 14: 다듬기 + 최종 마감

B-4. 전체를 한 장으로 (완성했을 때 그림)

```
        [Streamlit 화면]  ← Day 12~14 (강사가 보는 곳)
              │  버튼 클릭
              ▼
        [우리 FastAPI 서버]  ← Day 1~5 (이미 완성)
         ├─ children API
         ├─ lesson_records API
         └─ feedbacks API ─┐
                           ▼
              [프롬프트 조립] ← Day 4
```

```
        │  + RAG로 참고자료 끼움 ← Day 8~11
        ▼
  [Gemini AI]  → 피드백 생성
        │
        ▼
   [DB 저장]  ← Day 5
```

즉 남은 일 = ① AI를 똑똑하게(RAG) + ② 껍데기 씌우기(Streamlit) + ③ 정리(테스트·문서). 뼈대(Day 1~5)는 이미 서 있어. 넌 3분의 1이 아니라, **제일 어려운 "구조 잡기"를 이미 끝낸 상태**야.

B-5. 다음에 앉으면 할 첫 행동 (3일 마감 계획)

- **1일차:** Day 6(통합 테스트) + Day 7(README) → 1주차 완전 종료 + 이 복습 문서 자가진단
- **2일차:** Day 8~9(RAG 개념 + 쪼개기·임베딩·검색) → 제일 무거운 날
- **3일차:** Day 10~11(RAG 연결) + Day 12~14(Streamlit) → 마감

> 시작 문장: **"의자 앉기 → 화면 열기 → 이 문서 A-5 자가진단 3개만 말해보기 → Day 6 첫 클릭."**